

PHASE_8C_INTEGRATION

Phase 8C Integration Map — Attestation, E2EE/TEE, and Marketplace Seams

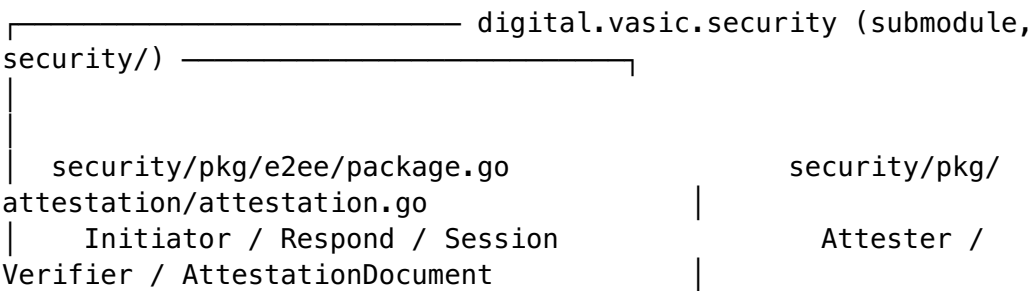
Revision: 1 **Last modified:** 2026-06-03 **Maintainer:** Helix Cluster OS —
Architecture Status: active

Scope. This document is the **code-grounded** integration map for the three Phase 8C seams described conceptually in docs/research/phase_08C/PHASE_8C_ROADMAP.md §5.1/§5.2/§5.3. It does **not** restate the roadmap. Every box and arrow below maps to a file:symbol that was opened and confirmed in the working tree. Where a seam is only **partially** wired in code (an interface exists but the production package is not yet consumed), it is labelled **PLANNED** rather than implemented, per CLAUDE-3.

Legend: [IMPLEMENTED] = the cited symbol exists and is exercised by code in this repo today. [PLANNED] = the seam (interface / label) exists, but the wiring to the named production package is **not present** in code yet — do not read these arrows as live integration.

0. Module-boundary map (decoupled submodule vs main-repo consumers)

Phase 8C splits security primitives into the **digital.vasic.security** Git submodule (module path `digital.vasic.security`, mounted at `security/` in this repo) and keeps consumers + a software GPU-attestation package in the main repo. The submodule is project-agnostic and reusable; the main repo holds the schedulers/marketplace/router that *would* consume it.



	Session.Seal / Session.Open		QuoteSigner (HW
seam)			
	(ML-KEM-768 + HKDF-SHA256 + AES-256-GCM)		(Ed25519
software reference; TDX/NVIDIA = HW seam)			
	security/pkg/gpuattest/{package.go, seal.go}		(PoVW + device-
sealed encrypt – submodule copy)			

▲		▲	
	[PLANNED] not imported		[PLANNED] not
imported	[PLANNED] not imported		
	MAIN REPO		
consumers			
	pkg/scheduler – AttestationGate (own Attestor iface; NOT the		
submodule) [§5.1]			
	pkg/gpuattest – Descriptor / Verifier / EnumerateNode /		
VerifyNode (main-repo software PoVW)			
	pkg/smartrouter – StrategyTEE routes by a Model.TEE bool		
ONLY [§5.2]			
	pkg/inference – Router/Backend (no e2ee/attestation		
reference) [§5.2]			
	pkg/marketplace – ChutesAdapter / CompositeScorer /		
GPUReservationTable /			
	Encryptor+Attestor seams (own AES-GCM/HMAC		
refs) [§5.3]			

Honest boundary fact (verified): no main-repo consumer package currently imports `digital.vasic.security/...`. Each consumer instead defines its **own** in-package seam interface (`scheduler.Attestor`, `marketplace.Encryptor`, `marketplace.Attestor`) backed by a stdlib software reference. The submodule’s PQ/TEE implementations are designed to drop in behind those exact interfaces; that substitution is **PLANNED**, not yet wired. This is called out in the package docs themselves — see `pkg/scheduler/attestation.go` lines 27–34 (“does NOT import the `digital.vasic.security` submodule”) and `pkg/marketplace/confidential.go` lines 13–18 / 28–34 (“The production `digital.vasic.security` submodule ... implements this SAME interface”).

5.1 Attestation → Scheduler

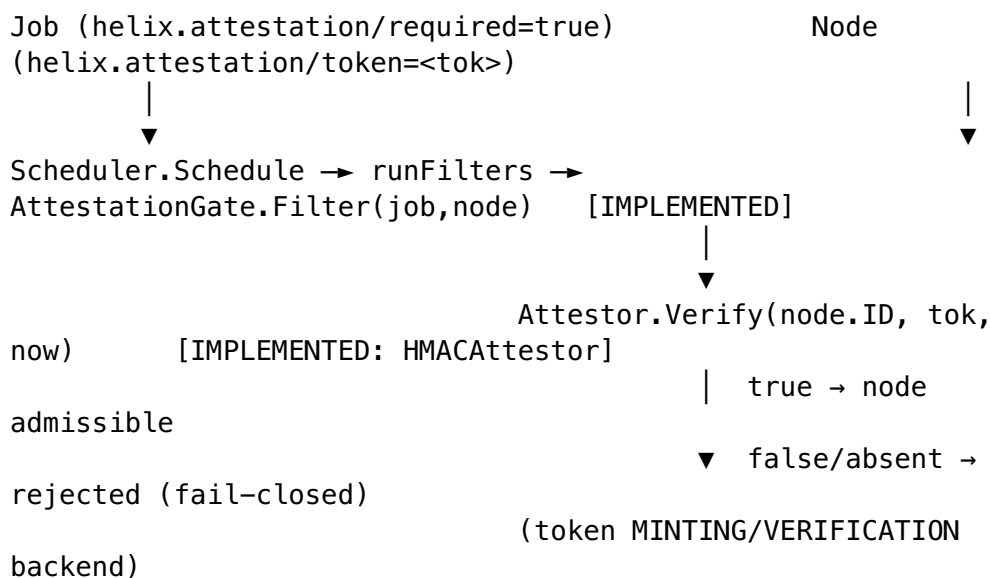
Goal: an attestation result gates scheduler admission so only attested nodes can host jobs that demand a trusted execution environment.

What is IMPLEMENTED today

pkg/scheduler/attestation.go defines **AttestationGate**, a real scheduler Plugin (it implements Name/Filter/Score/Bind from pkg/scheduler/plugins.go). It is consumed by the scheduler core: Scheduler.AddPlugin registers plugins and Scheduler.Schedule → runFilters calls each plugin's Filter (pkg/scheduler/scheduler.go, AddPlugin line 54, Schedule/runFilters from line 101). So the **admission predicate seam itself is live**.

The admission contract (AttestationGate.Filter, attestation.go lines 119–135):

- A job is marked attestation-required via SetJobRequireAttestation → label helix.attestation/required (LabelJobRequireAttestation, line 51).
- A node presents a token via SetNodeAttestationToken → label helix.attestation/token (LabelNodeAttestationToken, line 56).
- Filter admits an attestation-required job **only** when the injected Attestor.Verify(node.ID, token, now) returns true; it **fails closed** (rejects every node) when no Attestor is configured.
- The in-repo reference HMACAttestor (Issue/Verify, lines 161–227) does genuine HMAC-SHA256 node-bound, time-bounded token verification (hmac.Equal constant-time compare); a tampered/expired/wrong-node token genuinely fails.



What is PLANNED (interface exists, production backend not wired)

The Attestor interface (attestation.go lines 63–69) is the seam where a real backend plugs in. The roadmap's intended sources are:

[PLANNED] security/pkg/attestation.Verifier.Verify(doc, nonce, policy, now) (TDX/NVIDIA TEE quote)
[PLANNED] pkg/gpuattest.VerifyNode(v, proofs, nowTick) + Verifier.Verify(...) (software PoVW)
| would implement scheduler.Attestor (or feed a node-trust label)

▼
scheduler.AttestationGate.Attestor

- security/pkg/attestation already provides the verification half: Verifier.Verify (attestation.go line 300) checks signature, trusted-signer, nonce, freshness and a MeasurementPolicy against an AttestationDocument produced by Attester.Attest (line 197). Hardware rooting is itself an explicit seam — QuoteSigner (line 146).
- pkg/gpuattest provides software proof-of-GPU: Descriptor/Fingerprint, Verifier.Verify (attest.go line 161) with distinct sentinels (ErrExpired/ErrReplay/ErrTampered/ErrWrongKey/ErrForgedDescriptor), and node-level EnumerateNode/VerifyNode (multigpu.go lines 52/88).

Split state of the roadmap §5.1 “node-trust label / pkg/resources verified/ attested flag” arrow — the flag exists, its producer does not:

- **[IMPLEMENTED] resources-side flag + API.** pkg/resources *does* carry an attested flag today: GPUInfo.Attested bool (pkg/resources/types.go line 42) with accessors SetAttested / IsAttested (pkg/resources/gpuclass.go lines 170 / 176). EffectiveMultiplier (gpuclass.go line 159) and its doc comment reference the flag — the attested vs. unattested distinction is surfaced via IsAttested so a policy can refuse to schedule on unattested hardware. So the *destination* of the arrow (the trust flag and its read/write API) is real and exercised by code today.
- **[PLANNED] producer wiring.** What is **not yet present in code** is any adapter that makes security/pkg/attestation.Verifier or pkg/gpuattest.VerifyNode satisfy scheduler.Attestor, and any code path that *populates* GPUInfo.Attested (via SetAttested) or sets a node-trust label **from an attestation result**. Nothing calls SetAttested off the back of a verified quote; the flag is currently only settable by hand, never driven by the attestation pipeline.

The roadmap §5.1 arrow is therefore **half-wired**: the pkg/resources attested flag/API is **IMPLEMENTED**, but its *population from attestation* is **PLANNED**.

5.2 E2EE → Orchestrator/TEE (confidential LLM routing)

Goal: a tenant’s prompt is sealed in a PQ E2EE envelope that terminates only inside an attested enclave, so relays/routers see ciphertext + routing metadata only.

What is IMPLEMENTED today

security/pkg/e2ee/package.go provides a **complete, working PQ envelope**:

- Handshake: NewInitiator (line 103) publishes an ephemeral ML-KEM-768 encapsulation key; Respond (line 121) encapsulates → (ciphertext, Session); Initiator.Complete (line 140) decapsulates to the same shared secret.

- Key schedule: `deriveSessionKey` (line 163) = HKDF-SHA-256 over the shared secret, salted by KEM-ciphertext bytes, with a domain-separation info label.
- Record protection: `Session.Seal / Session.Open` (lines 281/295) = AES-256-GCM with monotonic/random nonce and explicit `ErrNonceReuse` rejection. `SealWithKey/OpenWithKey` (lines 366/379) are the stateless form.

This is the envelope a confidential inference path would carry.

What is PLANNED (the termination-in-enclave wiring does NOT exist)

The “terminates inside an attested enclave for confidential LLM routing” arrow is **PLANNED**. The closest implemented routing symbol is `pkg/smartrouter/smartrouter.go StrategyTEE → routeTEE` (lines 138/194), which restricts the candidate pool to models whose `Model.TEE bool` (line 71) is set, then picks the cheapest (`routeCost`).

```
tenant prompt
  | [PLANNED] e2ee.Session.Seal(plaintext) – envelope not
  | invoked by any router today
  ▼
pkg/smartrouter.Route(StrategyTEE) → routeTEE: keep models where
Model.TEE==true → routeCost [IMPLEMENTED]
  |
  | [PLANNED] envelope terminates inside enclave verified by
  | security/pkg/attestation.Verifier.Verify(...)
  ▼
pkg/inference.Router / Backend (no e2ee / attestation symbol
referenced) [IMPLEMENTED router, no E2EE]
```

Honest gap (verified): `Model.TEE` is a plain boolean trust *assertion* — `routeTEE` does not verify any attestation quote and does not decrypt/terminate any `e2ee` envelope. `pkg/inference (router.go/backend.go)` references neither `e2ee` nor attestation. Treating the TEE bool as a trust signal without a verified quote is exactly the PASS-bluff the roadmap §7 risk table warns about; this document records it as PLANNED so the doc does not imply non-existent confidential-routing wiring (CLAUDE-1 / CLAUDE-3).

5.3 Marketplace: registration → placement → metering → settlement

Goal: providers register capacity, the cluster places work cost-optimally, usage is metered against per-class capacity, and rewards settle over Helix Raft reputation.

What is IMPLEMENTED today (pkg/marketplace)

```

                                registration                                placement
External provider → ChutesAdapter.ListOffers →
CompositeScorer.Rank(offers, w, requireTEE)
```

```

(OfferSource.Fetch)    (package.go:116, stamps    (scorer.go:67 –
weighted normalised score;
                        Offer.Provider)
TEEMultiplier=1.5 promotes Confidential offers)
                                                                |
                                                                ▼ best-
first []ScoredOffer
                        metering                                confidential
dispatch (optional)
GPUReservationTable.Reserve(class,count) ←————
Encryptor.Seal / Attestor.Verify
(reservation.go:85 – per-class capacity;
(confidential.go: AESGCMEncryptor:66/75,
  ErrUnknownClass / ErrInsufficientCapacity;
HMACAttestor.Attest/Verify:115/122)
Reservation.Release:105 idempotent return)

```

- **Registration:** ChutesAdapter (package.go) implements Adapter; ListOffers (line 116) fetches via the injectable OfferSource and stamps each Offer.Provider so downstream ranking/audit can attribute the offer. The Offer struct carries the scored dimensions + a Confidential (TEE) flag (lines 54–70).
- **Placement:** CompositeScorer.Rank (scorer.go line 67) ranks offers by a weighted, per-dimension-normalised composite (DefaultWeights, line 18) with a TEEMultiplier (line 23) applied to Confidential offers when requireTEE is set; ties break deterministically by Offer.ID.
- **Metering / capacity accounting:** GPUReservationTable (reservation.go) enforces per-class capacity — Reserve (line 85) denies cross-class over-use (ErrUnknownClass, ErrInsufficientCapacity) and Reservation.Release (line 105) is idempotent. This is real accounting, concurrency-safe under a mutex.
- **Confidential path seams:** Encryptor (AES-256-GCM AESGCMEncryptor) and Attestor (HMAC-SHA256 HMACAttestor) in confidential.go are working stdlib references that genuinely round-trip / verify.

What is PLANNED

- **Cost-aware placement onto cluster nodes** (as opposed to ranking external offers) lives separately in pkg/scheduler/cost_gpu.go (CostAwareGPUPlacement.Score, line 66 — prefers the cheapest adequate node, reading cost_per_gpu_hour/cost_per_hour labels). The marketplace scorer and the scheduler cost plugin are **not** cross-wired today; the roadmap §5.3 “placement → pkg/scheduler” arrow is therefore **PLANNED** as a single flow.
- **Settlement over Raft reputation:** there is **no** Raft-reputation or payout/settlement code in pkg/marketplace (no audit subpackage exists; Offer has no settlement field). The roadmap §5.3 “settlement → Helix Raft reputation” and “metering → pkg/marketplace/audit commit-then-prove” arrows are **PLANNED**. Only capacity *accounting* (reservation table) is present.

[PLANNED] ScoredOffer (placement decision) → pkg/
scheduler.CostAwareGPUPlacement (not cross-wired)

[PLANNED] metered usage → pkg/marketplace/audit (commit-then-
prove) (subpackage absent)

[PLANNED] reward
reputation

→ Helix Raft

(no code present)

Cited file:symbol index (every claim is grounded)

Seam	File	Symbols (verified)	State
§5.1	pkg/scheduler/attestation.go	AttestationGate, Filter (L119), Attestor iface (L63), HMACAttestor.Issue/Verify (L183/L193), labels helix.attestation/required (L51) /token (L56)	IMPLEMENTED (own seam)
§5.1	pkg/scheduler/scheduler.go	Scheduler.AddPlugin (L54), Schedule/runFilters (L101+)	IMPLEMENTED
§5.1	pkg/scheduler/plugins.go	Plugin interface (Name/Filter/Score/Bind)	IMPLEMENTED
§5.1	security/pkg/attestation/attestation.go	Verifier.Verify (L300), Attester.Attest (L197), QuoteSigner (L146), AttestationDocument.SigningPayload (L101)	IMPLEMENTED in submodule; consumption PLANNED
§5.1	pkg/gpuattest/attest.go	Descriptor/Fingerprint (L37/L67), Verifier.Verify (L161), sentinels ErrExpired/ErrReplay/ErrTampered/ErrWrongKey/ErrForgedDescriptor	IMPLEMENTED; consumption PLANNED
§5.1	pkg/gpuattest/multigpu.go	EnumerateNode (L52), VerifyNode (L88)	IMPLEMENTED; consumption PLANNED
§5.1	pkg/resources/types.go, gpuclass.go	GPUInfo.Attested (types.go L42), SetAttested/IsAttested (gpuclass.go L170/L176), EffectiveMultiplier (gpuclass.go L159)	IMPLEMENTED (flag + API); population-from- attestation PLANNED
§5.2	security/pkg/e2ee/package.go	NewInitiator (L103), Respond (L121), Initiator.Complete (L140), Session.Seal/Open (L281/L295), deriveSessionKey (L163)	IMPLEMENTED in submodule
§5.2	pkg/smartrouter/smartrouter.go	StrategyTEE (L36), routeTEE (L194), Model.TEE (L71)	IMPLEMENTED (bool-only); quote/envelope wiring PLANNED
§5.2	pkg/inference/router.go, backend.go	Router, Backend (no e2ee/attestation reference)	IMPLEMENTED router; E2EE termination PLANNED
§5.3	pkg/marketplace/package.go	Adapter, ChutesAdapter.ListOffers (L116), Offer (+Confidential)	IMPLEMENTED

Seam	File	Symbols (verified)	State
§5.3	pkg/ marketplace/ scorer.go	CompositeScorer.Rank (L67), Weights/ DefaultWeights (L18), TEEMultiplier (L23)	IMPLEMENTED
§5.3	pkg/ marketplace/ reservation.go	GPUReservationTable.Reserve (L85), Reservation.Release (L105), ErrUnknownClass/ ErrInsufficientCapacity	IMPLEMENTED
§5.3	pkg/ marketplace/ confidential.go	Encryptor/AESGCMEncryptor.Seal/ Open (L66/L75), Attestor/ HMACAttestor.Attest/Verify (L115/ L122)	IMPLEMENTED (software refs)
§5.3	pkg/scheduler/ cost_gpu.go	CostAwareGPUPlacement.Score (L66), labels cost_per_gpu_hour/ cost_per_hour	IMPLEMENTED; cross-wire to marketplace PLANNED

Cross-reference: for the conceptual sub-phase plan and source-repo lineage, see docs/research/phase_08C/PHASE_8C_ROADMAP.md (§5.1/§5.2/§5.3, §6 priority, §9 phase bridge). This document is its code-grounded counterpart and supersedes none of it.